

Sujet d'examen UE NFE204 - Session 2

Année universitaire 2024-2025

Examen session hybride : 15 avril 2025

Consignes

Durée de l'examen : 3h00; Calculatrice autorisée ; Documents autorisés : 5 pages recto-verso de note personnelles. Ce sujet contient 3 pages.

Première partie : modélisation semi-structurée (8 pts)

Un grand magasin enregistre tous les tickets de caisse des transactions effectuées par ses clients. Un ticket de caisse comprend la date et l'heure de chaque transaction, le numéro de la caisse, le prix total, et la liste des articles achetés avec, pour chacun, la quantité et le prix unitaire. Le ticket de caisse comprend également l'identifiant du client si ce dernier a utilisé sa carte de fidélité. Sinon cet identifiant est à null.

1. Dans une base relationnelle, combien de tables faudrait-il pour représenter les informations d'un ticket de caisse (expliquer et donner le schéma de ces tables); combien d'insertions de ligne faut-il faire pour chaque ticket ?
2. Si on choisit une base NoSQL, comment représenter un ticket de caisse en JSON. Donnez un exemple simple. Combien d'insertions faut-il faire ?
3. Je veux compter le nombre de transactions par caisse et par jour. Quelle est la requête SQL si les données sont dans une base relationnelle ?
4. Même question si la base est NoSQL : comment calculer le nombre de transactions par caisse et par jour
5. Proposez un format de document semi-structuré (en JSON par exemple) pour une base des articles, la description de chaque article incluant la liste des ventes (une vente = date, quantité, client)

Solution :

1. En relationnel, il faut 2 tables, une pour les tickets avec les champs noCaisse, date, heure et prixTotal, une autre pour les articles avec idArticle, quantité et prixUnitaire. Un champ idClient dans la table Ticket peut être à null. La table Article comprend une clé étrangère idTicket pour savoir de quel ticket l'achat de l'article fait partie.

Il faut une insertion dans la table Ticket, et autant d'insertions dans la table Article qu'il y a d'articles achetés dans une même transaction.

2. En JSON, par exemple :

```
{
  "_id": 978,
  "noCaisse": 12,
  "date": "01/01/2017",
  "heure": "13:30",
  "prixTotal": 58.47,
  "idClient": null,
```

```

    "articles": [
      {"idArticle": 1982, "quantité": 5, "prix": 10.45},
      {"idArticle": 34, "quantité": 1, "prix": 45},
      {"idArticle": 154, "quantité": 3, "prix": 3.02},
    ]
  }

```

Une insertion suffit.

3. En SQL :

```

select count(*) from Ticket t, Article a
where a.idTicket = t.id
group by date, noCaisse

```

4. En NoSQL, il faut écrire et exécuter un programme MapReduce.

La fonction de Map émet une paire intermédiaire dont la clé est le critère de regroupement, soit la paire (date, numéro de caisse).

```

function fonctionMap ($ticket)
{
  emit ( [$ticket.date, $ticket.noCaisse], 1)
}

```

La fonction de Reduce effectue le compte des paires intermédiaires, autrement dit des tickets ayant même date et même numéro de caisse.

```

function fonctionReduce ($clé, $valeurs)
{
  return ($clé, count($valeurs))
}

```

Deuxième partie : MapReduce (7 pts)

Je veux calculer le montant moyen des tickets par client, pour les clients dont je connais l'identifiant.

1. Décrivez la modélisation MapReduce de ce calcul. Donnez le pseudo-code de chaque fonction, ou indiquez par un texte clair son déroulement.
2. Donnez la forme de la chaîne de traitement (*workflow*), avec des opérateurs Pig ou Spark, qui implante ce calcul.
3. Indiquez comment on produit un document 'Article' avec la liste des ventes à partir de la collection des tickets de caisse.

Solution :

1. La fonction de Map ne prend en compte que les tickets ayant un numéro de client non nul, et émet une paire intermédiaire dont la clé est le critère de regroupement, soit le numéro de client, et la valeur le prix total du ticket.

```

function fonctionMap ($ticket)
{
  if ($ticket.idClient not null) then
    emit ( $ticket.idClient, $ticket.prixTotal)
  end if
}

```

La fonction de Reduce effectue la moyenne des prix.

```
function fonctionReduce ($idClient, $listePrix)
{
    return ($idClient, avg($listePrix))
}
```

2. En Pig :

```
tickets_client = filter tickets BY idClient not null;
tickets_groupes = group tickets_client by idClient;
avg_par_client = foreach tickets_groupes generate group, AVG(tickets_groupes.prixTotal)
```

Troisième partie : systèmes distribués (3 pts)

On vous demande d'expliquer quelques-uns des principes du *consistent hashing*.

1. Expliquez comment on choisit le serveur sur lequel on place un document.
2. Pourquoi la structure est-elle basée sur un anneau ? Pourquoi pas un segment de droite par exemple ?
3. Pourquoi y a-t-il autant de valeurs sur l'anneau (2 puissance 64 typiquement) ? Pourquoi ne pas commencer avec 8 ou 16 valeurs, et augmenter ensuite ?
4. Quelle est l'utilité de la notion de nœud virtuel ?

Solution :

1. Chaque serveur est responsable d'un secteur sur l'anneau. L'identifiant d'un document est haché et la valeur de hachage détermine donc le serveur d'accueil.
2. Le structure d'anneau évite d'avoir à gérer le cas des extrémités d'une structure linéaire.
3. Comme expliqué en cours, on ne peut pas modifier la fonction de hachage. Autant donc choisir dès le départ une fonction maximale et immuable. La règle d'affectation sur l'anneau fournit l'adaptation au nombre de serveurs.
4. Les nœuds virtuels améliorent la distribution en tendant vers une nombre moyen de document sur chaque serveur

Quatrième partie : questions de cours (2 pts)

Pourquoi faut-il stocker les paires intermédiaires sur disque dans un processus MapReduce ? Pourquoi ne peut-on directement les transmettre aux machines qui effectuent la phase de Reduce ?